

Design Patterns and Principles

Exercise 1 & Exercise 2 — Complete Java Solutions

Deep Skilling Handbook — DotNet FSE Angular

This document provides complete, production-quality Java implementations for the Singleton Pattern and the Factory Method Pattern. Each exercise includes the full project structure, all Java source files with detailed comments, expected output, and key design principles.

Contents:

Exercise 1 — Singleton Pattern (Logger Utility) — Project: SingletonPatternExample

Exercise 2 — Factory Method Pattern (Document Management) — Project:

FactoryMethodPatternExample

Exercise 1: Implementing the Singleton Pattern

Scenario: You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

What is the Singleton Pattern?

The Singleton Pattern is a Creational Design Pattern that restricts instantiation of a class to exactly one object. It provides a global access point to that single instance. Common use cases include: logging utilities, configuration managers, thread pools, caches, and database connection pools.

UML Structure:

Logger — private static instance, private constructor, public static getInstance().

Client — calls Logger.getInstance() and uses the returned object.

Project Structure:

```
SingletonPatternExample/  
  src/  
    LogLevel.java      (Enum for log levels)  
    Logger.java        (Singleton class - thread-safe, lazy init)  
    SingletonTest.java (Test / Main class)  
  README.md
```

File 1: LogLevel.java

```
// LogLevel.java  
// Enum defining logging severity levels
```

```
public enum LogLevel {  
    INFO, DEBUG, WARNING, ERROR  
}
```

File 2: Logger.java (Core Singleton Class)

```
// Logger.java  
// Singleton Pattern - Thread-Safe using double-checked locking
```

```
import java.time.LocalDateTime;  
import java.time.format.DateTimeFormatter;  
import java.io.FileWriter;  
import java.io.IOException;  
import java.io.PrintWriter;
```

```
public class Logger {  
  
    // Step 1: Private static instance (volatile for thread visibility)  
    private static volatile Logger instance = null;  
  
    private int      logCount = 0;  
    private String   logFilePath;  
    private PrintWriter fileWriter;  
    private static final DateTimeFormatter FORMATTER =  
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");  
  
    // Step 2: Private constructor - prevents external instantiation  
    private Logger() {  
        logFilePath = "application.log";  
        System.out.println("[Logger] Logger instance CREATED (appears only once)");  
        try {  
            fileWriter = new PrintWriter(new FileWriter(logFilePath, true));  
        } catch (IOException e) {  

```

```

        System.err.println("[Logger] Could not open log file: " + e.getMessage());
    }
}

// Step 3: Public static getInstance() - Double-Checked Locking
// Thread-safe without locking on every call (performance optimized)
public static Logger getInstance() {
    if (instance == null) { // First check (no lock)
        synchronized (Logger.class) {
            if (instance == null) { // Second check (with lock)
                instance = new Logger();
            }
        }
    }
    return instance;
}

// Core logging methods
public void log(LogLevel level, String message) {
    String timestamp = LocalDateTime.now().format(FORMATTER);
    String entry = String.format("[%s] [%-7s] %s", timestamp, level, message);
    System.out.println(entry);
    if (fileWriter != null) {
        fileWriter.println(entry);
        fileWriter.flush();
    }
    logCount++;
}

public void info(String message) { log(LogLevel.INFO, message); }
public void debug(String message) { log(LogLevel.DEBUG, message); }
public void warning(String message) { log(LogLevel.WARNING, message); }
public void error(String message) { log(LogLevel.ERROR, message); }

public int getLogCount() { return logCount; }

public void printStats() {
    System.out.println("\n--- Logger Stats ---");
    System.out.println("Total entries logged : " + logCount);
    System.out.println("Log file : " + logFilePath);
    System.out.println("Instance hash code : " + this.hashCode());
}

// Prevent cloning of the singleton
@Override
protected Object clone() throws CloneNotSupportedException {
    throw new CloneNotSupportedException("Cloning of Logger is not allowed.");
}

public void close() {
    if (fileWriter != null) {
        fileWriter.close();
        System.out.println("[Logger] Log file closed.");
    }
}
}

```

File 3: SingletonTest.java (Main / Test Class)

```
// SingletonTest.java
// Tests and demonstrates the Singleton Pattern with the Logger class

public class SingletonTest {

    public static void main(String[] args) throws InterruptedException {

        System.out.println("=== Singleton Pattern - Logger Test ===\n");

        // Test 1: Verify same instance is returned every time
        System.out.println("--- Test 1: Instance Identity Check ---");
        Logger logger1 = Logger.getInstance();
        Logger logger2 = Logger.getInstance();
        Logger logger3 = Logger.getInstance();

        System.out.println("logger1 hash: " + logger1.hashCode());
        System.out.println("logger2 hash: " + logger2.hashCode());
        System.out.println("logger3 hash: " + logger3.hashCode());
        System.out.println("logger1 == logger2 : " + (logger1 == logger2)); // true
        System.out.println("logger1 == logger3 : " + (logger1 == logger3)); // true

        // Test 2: All references log to the SAME instance
        System.out.println("\n--- Test 2: Shared Logging State ---");
        logger1.info("Application started successfully.");
        logger2.debug("User module initialized.");
        logger3.warning("Memory usage at 75%.");
        logger1.error("Failed to connect to payment gateway.");

        System.out.println("Log count via logger1: " + logger1.getLogCount());
        System.out.println("Log count via logger2: " + logger2.getLogCount()); // same
        System.out.println("Log count via logger3: " + logger3.getLogCount()); // same

        // Test 3: Thread-safety - multiple threads use the same instance
        System.out.println("\n--- Test 3: Thread-Safety Verification ---");
        Runnable task = () -> {
            Logger threadLogger = Logger.getInstance();
            threadLogger.info("Log from thread: " + Thread.currentThread().getName());
            System.out.println("  Thread " + Thread.currentThread().getName()
                + " -> hash: " + threadLogger.hashCode());
        };

        Thread t1 = new Thread(task, "Thread-A");
        Thread t2 = new Thread(task, "Thread-B");
        Thread t3 = new Thread(task, "Thread-C");
        t1.start(); t2.start(); t3.start();
        t1.join(); t2.join(); t3.join();

        // Test 4: Clone prevention
        System.out.println("\n--- Test 4: Clone Prevention ---");
        System.out.println("Clone is blocked by design (CloneNotSupportedException).");

        // Final Stats
        Logger.getInstance().printStats();
        Logger.getInstance().close();

        System.out.println("\n=== Test Complete ===");
    }
}
```

Expected Console Output:

```
=== Singleton Pattern - Logger Test ===
```

```
--- Test 1: Instance Identity Check ---
[Logger] Logger instance CREATED (appears only once)
logger1 hash: 1829164700
logger2 hash: 1829164700
logger3 hash: 1829164700
logger1 == logger2 : true
logger1 == logger3 : true

--- Test 2: Shared Logging State ---
[2025-06-17 10:00:01] [INFO   ] Application started successfully.
[2025-06-17 10:00:01] [DEBUG  ] User module initialized.
[2025-06-17 10:00:01] [WARNING] Memory usage at 75%.
[2025-06-17 10:00:01] [ERROR  ] Failed to connect to payment gateway.
Log count via logger1: 4
Log count via logger2: 4
Log count via logger3: 4

--- Test 3: Thread-Safety Verification ---
Thread-A -> hash: 1829164700
Thread-B -> hash: 1829164700
Thread-C -> hash: 1829164700

--- Test 4: Clone Prevention ---
Clone is blocked by design (CloneNotSupportedException).

--- Logger Stats ---
Total entries logged : 7
Instance hash code   : 1829164700
[Logger] Log file closed.

=== Test Complete ===
```

Key Design Points:

1. private static volatile Logger instance — volatile ensures all threads see the latest value.
2. private Logger() — no external class can call new Logger().
3. Double-Checked Locking — thread-safe without locking on every call.
4. Clone prevention — overriding clone() blocks a common workaround.
5. All three references (logger1, logger2, logger3) share the same hash = same object.

How to Compile and Run:

```
cd SingletonPatternExample/src
javac LogLevel.java Logger.java SingletonTest.java
java SingletonTest
```

Exercise 2: Implementing the Factory Method Pattern

Scenario: You are developing a document management system that needs to create different types of documents (Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

What is the Factory Method Pattern?

The Factory Method Pattern is a Creational Design Pattern that defines an interface for creating objects but lets subclasses decide which class to instantiate. It promotes the Open/Closed Principle — open for extension, closed for modification. Adding a new document type (e.g., CSV) requires only a new subclass, with zero changes to existing code.

UML Structure:

Document (interface) — open(), save(), close(), getInfo(), getMimeType()

WordDocument, PdfDocument, ExcelDocument — implement Document

DocumentFactory (abstract) — abstract createDocument() factory method

WordDocumentFactory, PdfDocumentFactory, ExcelDocumentFactory — concrete factories

DocumentManager — client using factories without knowing concrete types

Project Structure:

FactoryMethodPatternExample/

src/

Document.java	(Product interface)
WordDocument.java	(Concrete Product 1)
PdfDocument.java	(Concrete Product 2)
ExcelDocument.java	(Concrete Product 3)
DocumentFactory.java	(Abstract Creator)
WordDocumentFactory.java	(Concrete Creator 1)
PdfDocumentFactory.java	(Concrete Creator 2)
ExcelDocumentFactory.java	(Concrete Creator 3)
DocumentManager.java	(Client / Context class)
FactoryMethodTest.java	(Test / Main class)

README.md

File 1: Document.java (Product Interface)

```
// Document.java
// Product interface - defines the contract all document types must follow

public interface Document {

    // Opens the document for editing
    void open();

    // Saves the document to disk at the given file path
    void save(String filePath);

    // Closes the document and releases resources
    void close();

    // Returns a formatted summary of the document's properties
    String getInfo();

    // Returns the MIME type of this document format
    String getMimeType();
}
```

File 2: WordDocument.java (Concrete Product 1)

```
// WordDocument.java
// Concrete Product - Microsoft Word document (.docx)

public class WordDocument implements Document {

    private String title;
    private String author;
    private boolean isOpen;
    private int pageCount;

    public WordDocument(String title, String author) {
        this.title = title;
        this.author = author;
        this.isOpen = false;
        this.pageCount = 0;
    }

    @Override
    public void open() {
        isOpen = true;
        System.out.println("[WordDocument] Opening '" + title + "' in Microsoft Word...");
        System.out.println(" -> Rendering DOCX format with track-changes enabled.");
    }

    @Override
    public void save(String filePath) {
        if (!isOpen) {
            System.out.println("[WordDocument] Error: Document not open.");
            return;
        }
        pageCount++;
        System.out.println("[WordDocument] Saving '" + title
            + "' to: " + filePath + ".docx");
        System.out.println(" -> Word XML serialization complete. Pages: " + pageCount);
    }

    @Override
    public void close() {
        isOpen = false;
        System.out.println("[WordDocument] Closing '" + title + "'.");
    }
}
```

```

@Override
public String getInfo() {
    return String.format(
        "Word Document | Title: %s | Author: %s | Open: %b | Pages: %d",
        title, author, isOpen, pageCount);
}

@Override
public String getMimeType() {
    return "application/vnd.openxmlformats-officedocument.wordprocessingml.document";
}
}

```

File 3: PdfDocument.java (Concrete Product 2)

// PdfDocument.java

// Concrete Product - Portable Document Format (.pdf)

```

public class PdfDocument implements Document {

    private String title;
    private boolean isPasswordProtected;
    private boolean isOpen;

    public PdfDocument(String title, boolean passwordProtected) {
        this.title = title;
        this.isPasswordProtected = passwordProtected;
        this.isOpen = false;
    }

    @Override
    public void open() {
        isOpen = true;
        System.out.println("[PdfDocument] Opening '" + title + "' in PDF viewer...");
        if (isPasswordProtected)
            System.out.println(" -> Password required. Decryption in progress...");
        System.out.println(" -> PDF 1.7 format loaded successfully.");
    }

    @Override
    public void save(String filePath) {
        if (!isOpen) {
            System.out.println("[PdfDocument] Error: Document not open.");
            return;
        }
        System.out.println("[PdfDocument] Saving '" + title
            + "' to: " + filePath + ".pdf");
        System.out.println(" -> Flattening layers, embedding fonts.");
        if (isPasswordProtected)
            System.out.println(" -> Applying AES-256 encryption.");
    }

    @Override
    public void close() {
        isOpen = false;
        System.out.println("[PdfDocument] Closing '" + title + "'.");
    }

    @Override
    public String getInfo() {
        return String.format(
            "PDF Document | Title: %s | Protected: %b | Open: %b",
            title, isPasswordProtected, isOpen);
    }

    @Override

```



```
    public String getMimeType() { return "application/pdf"; }  
}
```

File 4: ExcelDocument.java (Concrete Product 3)

```
// ExcelDocument.java
// Concrete Product - Microsoft Excel spreadsheet (.xlsx)

public class ExcelDocument implements Document {

    private String title;
    private int sheetCount;
    private boolean hasMacros;
    private boolean isOpen;

    public ExcelDocument(String title, int sheetCount, boolean hasMacros) {
        this.title = title;
        this.sheetCount = sheetCount;
        this.hasMacros = hasMacros;
        this.isOpen = false;
    }

    @Override
    public void open() {
        isOpen = true;
        System.out.println("[ExcelDocument] Opening '" + title + "' in Microsoft Excel...");
        System.out.println(" -> Loading " + sheetCount + " worksheet(s).");
        if (hasMacros)
            System.out.println(" -> VBA macros detected.");
    }

    @Override
    public void save(String filePath) {
        if (!isOpen) {
            System.out.println("[ExcelDocument] Error: Document not open.");
            return;
        }
        String ext = hasMacros ? ".xlsm" : ".xlsx";
        System.out.println("[ExcelDocument] Saving '" + title
            + "' to: " + filePath + ext);
        System.out.println(" -> Calculating " + sheetCount + " sheet(s).");
    }

    @Override
    public void close() {
        isOpen = false;
        System.out.println("[ExcelDocument] Closing '" + title + "'.");
    }

    @Override
    public String getInfo() {
        return String.format(
            "Excel Document | Title: %s | Sheets: %d | Macros: %b | Open: %b",
            title, sheetCount, hasMacros, isOpen);
    }

    @Override
    public String getMimeType() {
        return hasMacros
            ? "application/vnd.ms-excel.sheet.macroenabled.12"
            : "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet";
    }
}
```

File 5: DocumentFactory.java (Abstract Creator)

```
// DocumentFactory.java
// Abstract Creator - defines the factory method and common operations
```

```

public abstract class DocumentFactory {

    // FACTORY METHOD - subclasses decide which Document type to create
    public abstract Document createDocument(String title);

    // Template method - uses the factory method then performs common steps
    public Document createAndPrepareDocument(String title, String savePath) {
        System.out.println("\n[DocumentFactory] Creating document: '" + title + "'");

        // Step 1: Create via factory method (polymorphic)
        Document doc = createDocument(title);

        // Step 2: Open and save (common steps for all document types)
        doc.open();
        doc.save(savePath + "/" + title.replaceAll("\\s+", "_"));

        System.out.println("[DocumentFactory] Ready: " + doc.getInfo());
        return doc;
    }

    // Returns the type of document this factory creates
    public abstract String getFactoryType();
}

```

File 6: WordDocumentFactory.java (Concrete Creator 1)

```

// WordDocumentFactory.java
public class WordDocumentFactory extends DocumentFactory {

    private String defaultAuthor;

    public WordDocumentFactory(String defaultAuthor) {
        this.defaultAuthor = defaultAuthor;
    }

    @Override
    public Document createDocument(String title) {
        System.out.println(" [WordDocumentFactory] Creating WordDocument...");
        return new WordDocument(title, defaultAuthor);
    }

    @Override
    public String getFactoryType() { return "Word Document Factory (.docx)"; }
}

```

File 7: PdfDocumentFactory.java (Concrete Creator 2)

```

// PdfDocumentFactory.java
public class PdfDocumentFactory extends DocumentFactory {

    private boolean enablePasswordProtection;

    public PdfDocumentFactory(boolean enablePasswordProtection) {
        this.enablePasswordProtection = enablePasswordProtection;
    }

    @Override
    public Document createDocument(String title) {
        System.out.println(" [PdfDocumentFactory] Creating PdfDocument...");
        return new PdfDocument(title, enablePasswordProtection);
    }

    @Override
    public String getFactoryType() { return "PDF Document Factory (.pdf)"; }
}

```

File 8: ExcelDocumentFactory.java (Concrete Creator 3)

```
// ExcelDocumentFactory.java
public class ExcelDocumentFactory extends DocumentFactory {

    private int    defaultSheetCount;
    private boolean enableMacros;

    public ExcelDocumentFactory(int defaultSheetCount, boolean enableMacros) {
        this.defaultSheetCount = defaultSheetCount;
        this.enableMacros      = enableMacros;
    }

    @Override
    public Document createDocument(String title) {
        System.out.println(" [ExcelDocumentFactory] Creating ExcelDocument...");
        return new ExcelDocument(title, defaultSheetCount, enableMacros);
    }

    @Override
    public String getFactoryType() { return "Excel Document Factory (.xlsx/.xlsm); }
}
```

File 9: DocumentManager.java (Client / Context Class)

```
// DocumentManager.java
// Client - works with DocumentFactory and Document interfaces only

import java.util.ArrayList;
import java.util.List;

public class DocumentManager {

    private List<Document> openDocuments = new ArrayList<>();
    private DocumentFactory currentFactory;

    public void setFactory(DocumentFactory factory) {
        this.currentFactory = factory;
        System.out.println("\n[Manager] Factory: " + factory.getFactoryType());
    }

    public Document createDoc(String title, String savePath) {
        if (currentFactory == null)
            throw new IllegalStateException("No factory set. Call setFactory() first.");
        Document doc = currentFactory.createAndPrepareDocument(title, savePath);
        openDocuments.add(doc);
        return doc;
    }

    public void closeAllDocuments() {
        System.out.println("\n[Manager] Closing all open documents...");
        for (Document doc : openDocuments) {
            doc.close();
        }
        openDocuments.clear();
    }

    public void printSummary() {
        System.out.println("\n[Manager] Open Documents (" + openDocuments.size() + "):");
        System.out.println(" " + "-".repeat(65));
        for (Document doc : openDocuments) {
            System.out.println(" " + doc.getInfo());
            System.out.println("  MIME: " + doc.getMimeType());
            System.out.println(" " + "-".repeat(65));
        }
    }
}
```

File 10: FactoryMethodTest.java (Main / Test Class)

```
// FactoryMethodTest.java
// Main test class - demonstrates the Factory Method Pattern

public class FactoryMethodTest {

    public static void main(String[] args) {

        System.out.println("=== Factory Method Pattern - Document Management Test ===");

        DocumentManager manager = new DocumentManager();

        // Test 1: Word Document Factory
        System.out.println("\n--- Test 1: Word Document Factory ---");
        manager.setFactory(new WordDocumentFactory("Alice Johnson"));
        manager.createDoc("Project Proposal", "/docs/word");
        manager.createDoc("Meeting Minutes", "/docs/word");

        // Test 2: PDF Document Factory (with password protection)
        System.out.println("\n--- Test 2: PDF Document Factory ---");
    }
}
```

```

manager.setFactory(new PdfDocumentFactory(true));
manager.createDoc("Annual Report 2025", "/docs/pdf");
manager.createDoc("User Manual", "/docs/pdf");

// Test 3: Excel Document Factory
System.out.println("\n--- Test 3: Excel Document Factory ---");
manager.setFactory(new ExcelDocumentFactory(4, false));
manager.createDoc("Q2 Sales Dashboard", "/docs/excel");

manager.setFactory(new ExcelDocumentFactory(2, true));
manager.createDoc("Automation Report", "/docs/excel");

// Print all open documents
manager.printSummary();

// Test 4: Open/Closed Principle demonstration
System.out.println("\n--- Test 4: Open/Closed Principle ---");
System.out.println("To add CsvDocument: create CsvDocument + CsvDocumentFactory.");
System.out.println("Zero changes to existing classes. OCP satisfied!");

// Cleanup
manager.closeAllDocuments();

System.out.println("\n=== Test Complete ===");
}
}

```

Expected Console Output:

=== Factory Method Pattern - Document Management Test ===

[Manager] Factory: Word Document Factory (.docx)

[DocumentFactory] Creating document: 'Project Proposal'

[WordDocumentFactory] Creating WordDocument...

[WordDocument] Opening 'Project Proposal' in Microsoft Word...

-> Rendering DOCX format with track-changes enabled.

[WordDocument] Saving 'Project Proposal' to: /docs/word/Project_Proposal.docx

-> Word XML serialization complete. Pages: 1

[DocumentFactory] Ready: Word Document | Title: Project Proposal | ...

[Manager] Factory: PDF Document Factory (.pdf)

[DocumentFactory] Creating document: 'Annual Report 2025'

[PdfDocumentFactory] Creating PdfDocument...

[PdfDocument] Opening 'Annual Report 2025' in PDF viewer...

-> Password required. Decryption in progress...

-> PDF 1.7 format loaded successfully.

[PdfDocument] Saving 'Annual Report 2025' to: /docs/pdf/Annual_Report_2025.pdf

-> Applying AES-256 encryption.

[Manager] Open Documents (6):

Word Document | Title: Project Proposal | Author: Alice Johnson | ...

MIME: application/vnd.openxmlformats-officedocument.wordprocessingml.document

Excel Document | Title: Automation Report | Sheets: 2 | Macros: true | ...

MIME: application/vnd.ms-excel.sheet.macroenabled.12

[Manager] Closing all open documents...

=== Test Complete ===

Key Design Points:

1. Open/Closed Principle (OCP) — adding CsvDocument needs 2 new classes, zero changes to existing

code.

2. Dependency Inversion — DocumentManager depends on Document interface, not concrete classes.
3. Single Responsibility — each factory class has one job: create its document type.
4. Template Method — createAndPrepareDocument() calls the factory method then does common steps.
5. Polymorphism — openDocuments is List of Document; all operations work via the interface.

How to Compile and Run:

```
cd FactoryMethodPatternExample/src
```

```
javac Document.java WordDocument.java PdfDocument.java ExcelDocument.java \  
    DocumentFactory.java WordDocumentFactory.java PdfDocumentFactory.java \  
    ExcelDocumentFactory.java DocumentManager.java FactoryMethodTest.java
```

```
java FactoryMethodTest
```

Pattern Comparison Summary

Singleton Pattern (Exercise 1)

Category : Creational

Intent : Ensure only one instance exists; provide global access

When to use : Shared resource — logger, config, DB pool, cache

Participants : Singleton class only

Key mechanism : Private constructor + static getInstance() + double-checked locking

Factory Method Pattern (Exercise 2)

Category : Creational

Intent : Delegate object creation to subclasses

When to use : Multiple product variants; decouple creation from usage

Participants : Product interface, Concrete Products, Abstract Creator, Concrete Creators

Key mechanism : Abstract createDocument() overridden in each concrete factory subclass

End of Design Patterns — Exercise 1 & Exercise 2 Solutions